

GML_05_01-sure.R

frank

2024-05-06

```
# feb 2023
# r script for simulation of CAPM and SURE
# written by Frank Lehrbass (->lehrbass.de) for teaching purposes only
# students are solely responsible for whatever they do with this coding
# this R coding is WITHOUT ANY WARRANTY

#data from example in book Finanzmarkt?konometrie CH 2

# Step 1

# Clean previous variables
rm(list=ls(all=TRUE))

# Load packages
library(tseries)

## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo

library(fUnitRoots)
#library(fts)
library(lmtest)

## Warning: Paket 'lmtest' wurde unter R Version 4.3.3 erstellt

## Lade nötiges Paket: zoo

##
## Attache Paket: 'zoo'

## Die folgenden Objekte sind maskiert von 'package:base':
##
##   as.Date, as.Date.numeric

library(sandwich)

## Warning: Paket 'sandwich' wurde unter R Version 4.3.3 erstellt

library(systemfit)

## Lade nötiges Paket: Matrix

## Lade nötiges Paket: car
```

```

## Warning: Paket 'car' wurde unter R Version 4.3.3 erstellt

## Lade nötiges Paket: carData

##
## Please cite the 'systemfit' package as:
## Arne Henningsen and Jeff D. Hamann (2007). systemfit: A Package for
## Estimating Systems of Simultaneous Equations in R. Journal of Statistical
## Software 23(4), 1-40. http://www.jstatsoft.org/v23/i04/.
##
## If you have questions, suggestions, or comments regarding the 'systemfit'
## package, please use a forum or 'tracker' at systemfit's R-Forge site:
## https://r-forge.r-project.org/projects/systemfit/

library(MASS)

#UPDATE HERE=====
INDEX_FILE = "GML_05_01 sure.csv"

#define in sample
START_DATE = "1995-12-31"
END_DATE = "2000-12-29"

#UPDATE ENDS=====

data_table <- read.table(INDEX_FILE, header=TRUE, sep=";",dec=",")
INDEX <- data_table[,2]
Dates <- data_table[,1]
DateStamp = as.Date(Dates,format="%d.%m.%Y")
zINDEX = zoo(INDEX,DateStamp)

# Generate sample

zDataIS <- window(zINDEX, start = START_DATE, end = END_DATE)

# Write data to file for Excel analysis and plot it, give intuitive names
indexReturn = diff(log(zDataIS[,1]))#book uses stetige Renditen, S. 42 Fn

# 2 Firmen Event Studie - Datensimulation -----

set.seed(1)

n=length(indexReturn)
volas = c(0.03,0.02)
means = c(0,0)
rho = 0.85
VOLS = diag(,2,2)
diag(VOLS) = volas
VOLS

```

```
##           [,1] [,2]  
## [1,] 0.03 0.00  
## [2,] 0.00 0.02  
  
RHO <- matrix(c(1,rho,rho,1),2,2)  
COV = t(VOLS) %*% RHO %*% VOLS  
COV  
  
##           [,1]      [,2]  
## [1,] 0.00090 0.00051  
## [2,] 0.00051 0.00040  
  
EE <- mvrnorm(n,means,COV)  
epsa = EE[,1]  
epsb = EE[,2]  
  
X=coredata(indexReturn) #this removes time stamps  
  
Ya = 1 * X + epsa  
Yb = 2 * X + epsb  
  
head(Ya)  
  
## [1] 0.121681603 -0.004247059 0.033020324 -0.039344027 0.001557505  
## [6] 0.032486926  
  
head(Yb)  
  
## [1] 0.1763536608 -0.0003302463 0.0200653877 -0.0141396263 0.0293825831  
## [6] 0.0282671403  
  
# 2 Firmen Event Studie - Ereignisbau -----  
  
#now event of xtra return of 10%/2% on day 30/40  
Ya[30]=Ya[30]+0.1  
Yb[40]=Yb[40]+0.02  
  
#create dummy  
Da = rep(0,60)  
Db=Da  
Da[30]=1  
Db[40]=1  
  
Da[1:30]  
  
## [1] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1  
  
# Step 2  
  
# 2 Firmen Event Studie - NAIV -----  
  
eq1 <- Ya~X+Da  
eq2 <- Yb~X+Db
```

#two separate OLS

`summary(lm(eq1))`

```
##
## Call:
## lm(formula = eq1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.057746 -0.017792  0.000531  0.017210  0.062473
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.002862   0.003534  -0.810   0.42150
## X             1.021027   0.052647  19.394 < 2e-16 ***
## Da           0.090559   0.026590   3.406  0.00122 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.02628 on 57 degrees of freedom
## Multiple R-squared:  0.8755, Adjusted R-squared:  0.8711
## F-statistic: 200.4 on 2 and 57 DF,  p-value: < 2.2e-16
```

`summary(lm(eq2))`

```
##
## Call:
## lm(formula = eq2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.030982 -0.009791 -0.000948  0.008082  0.051850
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.002523   0.002314  -1.090   0.280
## X             1.987918   0.034902  56.958 <2e-16 ***
## Db           0.013267   0.017628   0.753   0.455
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01724 on 57 degrees of freedom
## Multiple R-squared:  0.9833, Adjusted R-squared:  0.9827
## F-statistic: 1675 on 2 and 57 DF,  p-value: < 2.2e-16
```

Step 3

2 Firmen Event Studie - DIAG -----

#check correlation of resis of OLS

#H0: rho is zero

```

corResis = cor(residuals(lm(eq1)), residuals(lm(eq2)))
corResis

## [1] 0.8261268

#test as in assenmacher, 2002, 196, is based on resis of OLS

T = n #sample size
G = 2 #no of eqs

sure_stat = T*corResis^2
#critical
sure_df = G * (G-1)/2

sure_crit = qchisq(0.99, sure_df)
#compare
sure_stat

## [1] 40.94913

sure_crit

## [1] 6.634897

#hence reject H0 = time for SURE

# Step 4

# 2 Firmen Event Studie - SURE -----

myData = data.frame(cbind(Ya,Yb,X,Da,Db))
#as before but compact
fitOLS <- systemfit( list(eq1, eq2), method = "OLS", data=myData)
#a more adequate approach
fitSUR <- systemfit( list(eq1, eq2), method = "SUR", data=myData)
summary(fitSUR)

##
## systemfit results
## method: SUR
##
##          N  DF      SSR detRCov   OLS-R2 McElroy-R2
## system 120 114 0.056557      0 0.957444   0.981481
##
##      N DF      SSR      MSE      RMSE      R2  Adj R2
## eq1 60 57 0.039436 0.000692 0.026303 0.875320 0.870945
## eq2 60 57 0.017121 0.000300 0.017331 0.983094 0.982501
##
## The covariance matrix of the residuals used for estimation
##           eq1           eq2
## eq1 0.000690787 0.000374382
## eq2 0.000374382 0.000297299
##

```

```

## The covariance matrix of the residuals
##           eq1           eq2
## eq1 0.000691865 0.000380951
## eq2 0.000380951 0.000300364
##
## The correlations of the residuals
##           eq1           eq2
## eq1 1.000000 0.835668
## eq2 0.835668 1.000000
##
## SUR estimates for 'eq1' (equation 1)
## Model Formula: Ya ~ X + Da
##
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.00297176 0.00352109 -0.84399 0.4022
## X           1.01976826 0.05253147 19.41252 < 2.22e-16 ***
## Da           0.09849193 0.01498794 6.57141 1.6419e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.026303 on 57 degrees of freedom
## Number of observations: 60 Degrees of Freedom: 57
## SSR: 0.039436 MSE: 0.000692 Root MSE: 0.026303
## Multiple R-Squared: 0.87532 Adjusted R-Squared: 0.870945
##
## SUR estimates for 'eq2' (equation 2)
## Model Formula: Yb ~ X + Db
##
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.00267124 0.00230848 -1.15714 0.2520395
## X           1.98351963 0.03457848 57.36284 < 2.22e-16 ***
## Db           0.02678104 0.00993607 2.69533 0.0092253 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.017331 on 57 degrees of freedom
## Number of observations: 60 Degrees of Freedom: 57
## SSR: 0.017121 MSE: 3e-04 Root MSE: 0.017331
## Multiple R-Squared: 0.983094 Adjusted R-Squared: 0.982501

#For comparison
coeftest(fitOLS)

##
## t test of coefficients:
##
##           Estimate Std. Error t value Pr(>|t|)
## eq1_(Intercept) -0.0028616 0.0035343 -0.8097 0.419824
## eq1_X           1.0210271 0.0526470 19.3938 < 2.2e-16 ***
## eq1_Da           0.0905595 0.0265902 3.4058 0.000912 ***

```

```
## eq2_(Intercept) -0.0025229  0.0023140 -1.0903  0.277893
## eq2_X           1.9879178  0.0349017 56.9577 < 2.2e-16 ***
## eq2_Db          0.0132669  0.0176276  0.7526  0.453229
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
coeftest(fitSUR)
```

```
##
## t test of coefficients:
##
##              Estimate Std. Error t value  Pr(>|t|)
## eq1_(Intercept) -0.0029718  0.0035211 -0.8440  0.400443
## eq1_X           1.0197683  0.0525315 19.4125 < 2.2e-16 ***
## eq1_Da          0.0984919  0.0149879  6.5714 1.556e-09 ***
## eq2_(Intercept) -0.0026712  0.0023085 -1.1571  0.249632
## eq2_X           1.9835196  0.0345785 57.3628 < 2.2e-16 ***
## eq2_Db          0.0267810  0.0099361  2.6953  0.008096 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Step 5
```

```
# 2 Firmen Event Studie - SURE Diagnostik -----
```

```
#NOTE: Below only for eq1, for eq2 it is the same procedure ...
```

```
#systematic part of model, ie we can use ols if needed!!!-----
```

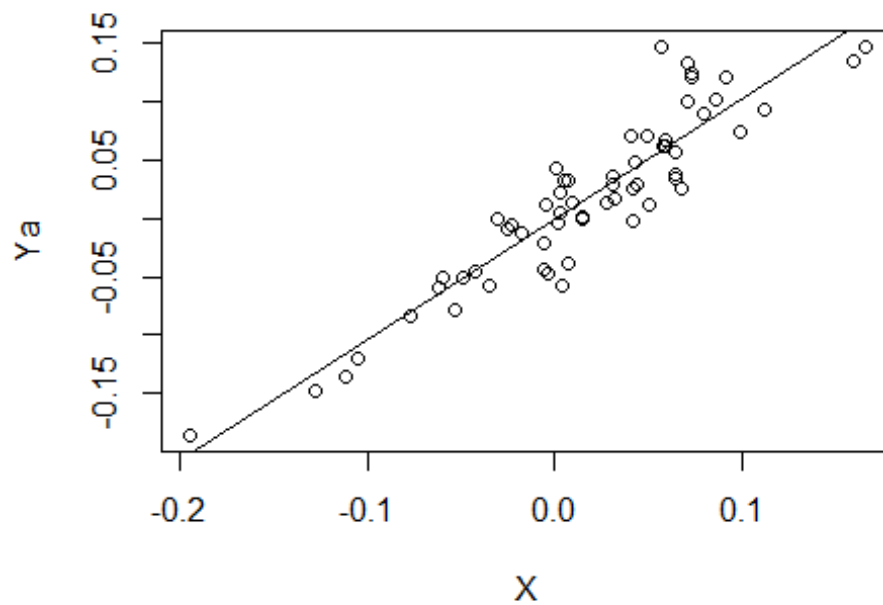
```
#Linearity?
```

```
x11()
```

```
regline = lm(Ya ~ X)
```

```
plot(Ya ~ X)
```

```
abline(regline)
```



```
#RESET Test von Ramsey  $H_0$  = Linearer Zusammenhang
resettest(regline,power=2,type="regressor")

##
## RESET test
##
## data: regline
## RESET = 0.17457, df1 = 1, df2 = 57, p-value = 0.6777

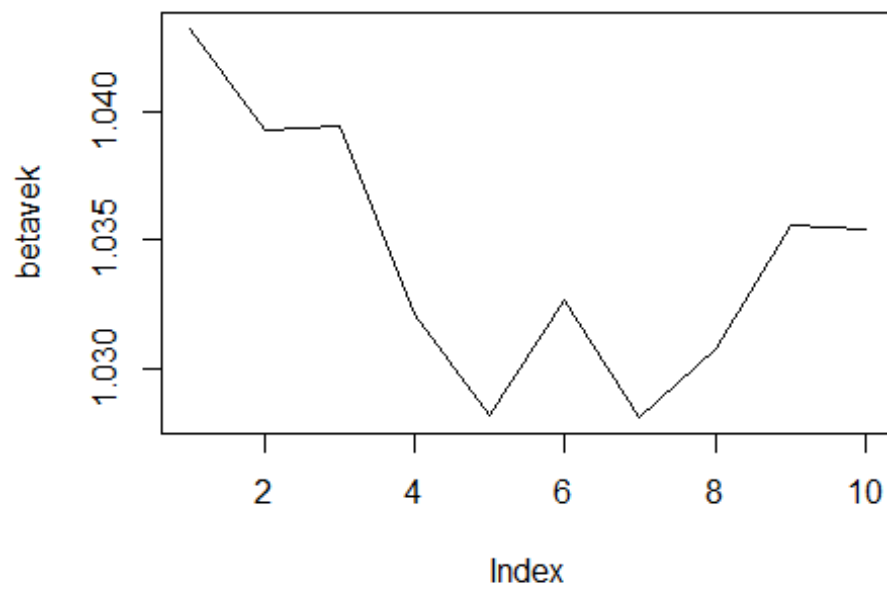
#multicoll, how should it?-----
cor(X,Da)#says it all

## [1] 0.0801516

vif(lm(eq1))

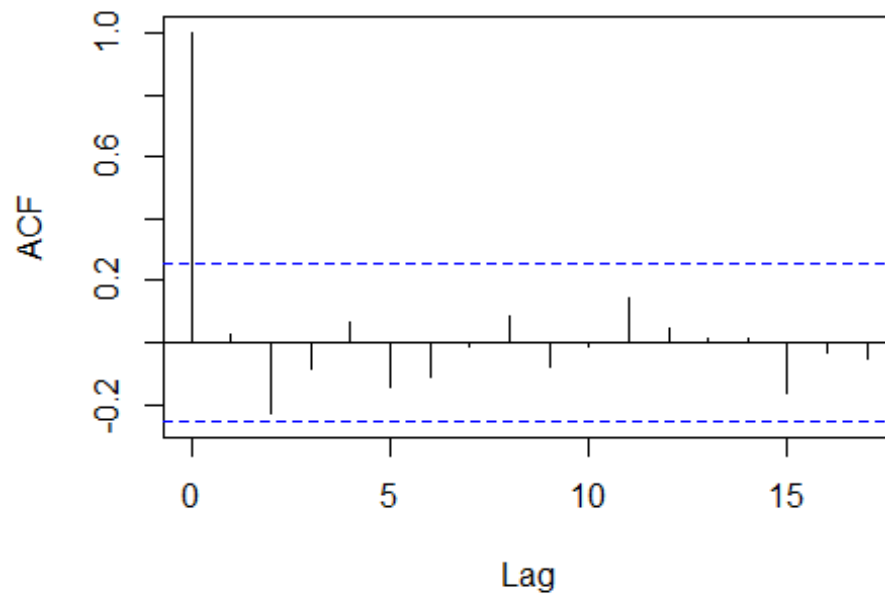
##          X          Da
## 1.006466 1.006466

#breaks?
betavek = rep(0,10)
for (i in c(1:10))
{
  betavek[i]=coef(lm(Ya[1:(50+i)]~X[1:(50+i)]))[2]
}
plot(betavek, type = 'l')
```

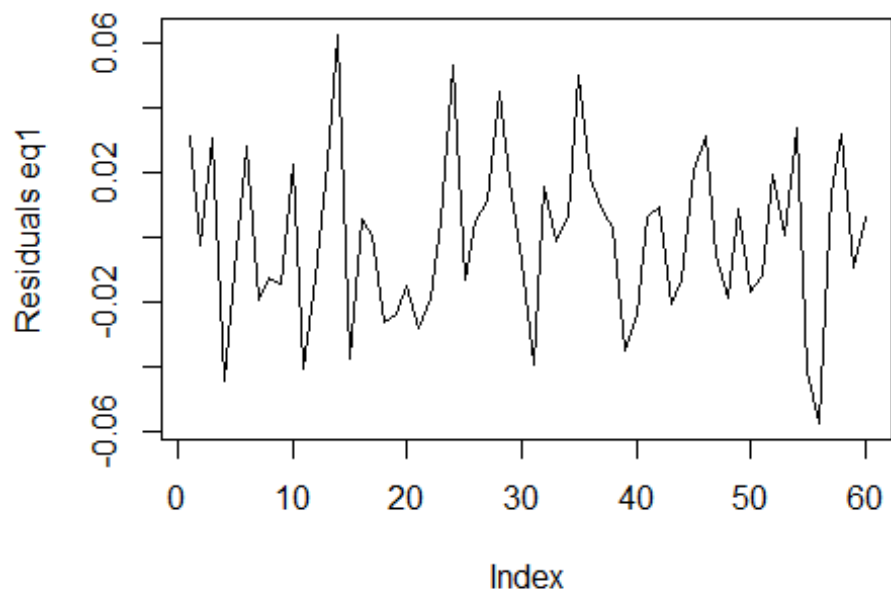


```
#stochastic part-----  
  
#Autokorr-----  
#analyze resis of sure, here eq1  
acf(residuals(fitSUR)$eq1, ci = 0.95) #etc
```

Series residuals(fitSUR)\$eq1



```
TT = length(Ya)
dw_SURE<-(sum((residuals(fitSUR)$eq1[-1]-residuals(fitSUR)$eq1[1:TT-
1])^2))/(sum((residuals(fitSUR)$eq1)^2))
dw_SURE
## [1] 1.923541
#use DW Tables if u want exact reading
#hetero-----
x11()
plot(residuals(fitSUR)$eq1, type='l', ylab = "Residuals eq1")
```



```
#White Test (handles non-normal terms, details see schröder book, p 115)
#H0: Homo
#aux reg

Xsq = X^2
Dsqr = Da^2
XD = X*Da
aux_reg = lm((residuals(fitSUR)$eq1)^2~X+Da+Xsq+Dsqr+XD)
summary(aux_reg)

##
## Call:
## lm(formula = (residuals(fitSUR)$eq1)^2 ~ X + Da + Xsq + Dsqr +
##      XD)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.0007711 -0.0005631 -0.0002539  0.0001745  0.0031155
##
## Coefficients: (2 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.0006652  0.0001366   4.870 9.52e-06 ***
## X            0.0027649  0.0017239   1.604   0.114
## Da          -0.0007313  0.0008702  -0.840   0.404
## Xsq         -0.0098171  0.0158830  -0.618   0.539
## Dsqr                NA         NA      NA      NA
## XD                NA         NA      NA      NA
## ---
```

```

## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.00086 on 56 degrees of freedom
## Multiple R-squared:  0.0592, Adjusted R-squared:  0.008795
## F-statistic: 1.175 on 3 and 56 DF,  p-value: 0.3277

#calc R² by feet
sigma_hat1 =
coefficients(aux_reg)[1]+coefficients(aux_reg)[2]*X+coefficients(aux_reg)[3]*
Da+
               coefficients(aux_reg)[4]*I(X^2)
#other terms na coeffs, I() = take as is, no interpretation

aux_R_sq = cor(sigma_hat1,(residuals(fitSUR)$eq1)^2)^2

#test stat
white_stat = n*aux_R_sq

#crit val
k = 5 #count above
white_df = k

white_crit = qchisq(0.99, white_df)
#compare, H0: Homo
white_stat

## [1] 3.551713

white_crit

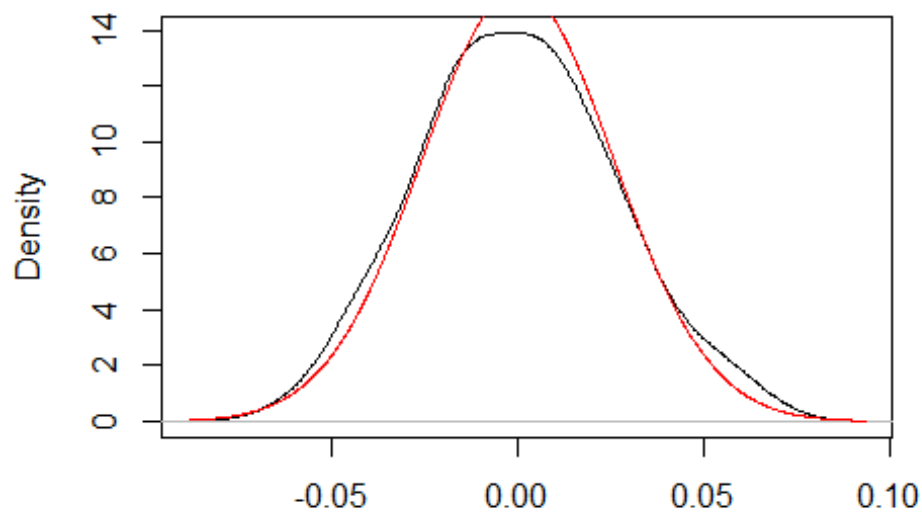
## [1] 15.08627

#hence keep H0

#normality-----
resi1 = residuals(fitSUR)$eq1
x11()#smooth it
kernel_den = density(resi1)
plot(kernel_den, main="Smoothed Residual Distribution", xlab = " ")
x <- seq(min(kernel_den$x),max(kernel_den$x),length = 1000)
#compare to normal
std_den = dnorm(x,mean = mean(resi1),sd = sd(resi1))
lines(x,std_den,type="l",col="red")

```

Smoothed Residual Distribution



```
shapiro.test(resi1)#H0: Is normal
```

```
##  
##  Shapiro-Wilk normality test  
##  
## data:  resi1  
## W = 0.99112, p-value = 0.9417
```

```
jarque.bera.test(resi1)#Nullhypothese ?Normalverteilung?
```

```
##  
##  Jarque Bera Test  
##  
## data:  resi1  
## X-squared = 0.47274, df = 2, p-value = 0.7895
```

```
#hence, keep H0
```

```
#BONUS: Event Study Old Style
```

```
#Unterstelle sogar Kenntnis des wahren Marktmodells, d.h. normal returns sind
```

```
NRa = 1 * X[30]
```

```
NRb = 2 * X[30]
```

```
ARa = Ya[30]-NRa
```

```
ARb = Yb[30]-NRb
```

```
AAR = (ARa+ARb)/2
```

```
t_stat<-sqrt(2)*AAR/sd(c(ARa,ARb))  
t_stat
```

```
## [1] 0.8019752
```

```
#crit value one sided
```

```
qnorm(0.95)
```

```
## [1] 1.644854
```

```
#Ende
```